

ThreadTest.kt has a data race in it (b.c.d).

Data race: informally, when 2 threads access same variable, one of them writes to it, and there's nothing in the code to ensure one happened before the other.

↳ in theory, the write and the read could happen "at the same time"

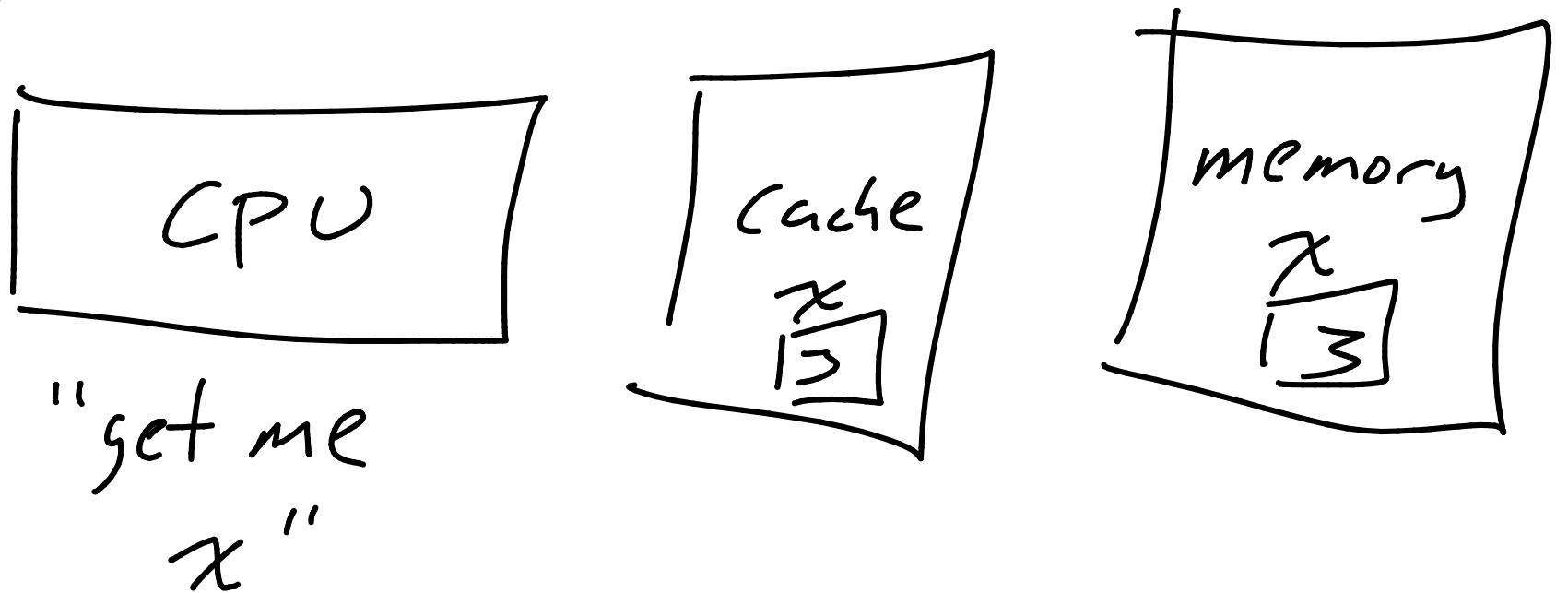
When a data race happens, lose guarantees that the lines of code you've written happen in some theoretical interleaving
- you lose almost everything

2 main challenges

① a good compiler reorders code to make things faster, if it can

guarantee output is the same
(analysis only within one thread)

② memory is copied into multiple places



Cache is small, but faster than DRAM
if you have multiple threads
and a data race, they may
see inconsistent versions

What most languages offer is a
guarantee that if use tools to
properly synchronize your code,
then some ordering of your
code as written happens.

Avoid data races with

- synchronized
- locks
- mutexes

-

- volatile →

requires that some interleaved version of the code that I see be consistent with the output



but doesn't

Solve "bad interleaving"
(could still have happened w/ volatile)

$x = \text{balance}$

$x = x + 10$

$\text{balance} = x$

$y = \text{balance}()$

,

$y = y + 10$

$\text{balance} = y$

lost \$10

As programmers, how do you avoid making these bugs?

- (0) Don't write code
- (1) Don't write buggy code
- (2) Track carefully shared variables and make sure you always lock them
- (3) Keep shared mem as localized as possible
- (4) Make an entire class doc'd as threadsafe (everything is synced) or not, and doc'd as such
- (5) Avoid global variables
- (6) Avoid multithreading when unnecessary
- (7) Use a threadsafe PL
- (8) Avoid shared variables
 - prefer parameters and return values

↑
scopes
objects
threads

- data race
 - bad interleaving
- } race condition
↓
behavior by a thread that depends on order that things happen in other threads

sophomoric - bug

Wikipedia - bug if it is one

"synchronized" is not for use in coroutines

most of the cool synchronization techniques don't work in coroutines

What's the problem?

"synchronized" could result in a thread blocking and waiting for something

For coroutines, we want that thread

to do some other task

- the work to do the swapping runs on that thread

—
Mutex class in Kotlin that gives you critical sections like synchronized or other locks

```
val m = Mutex()
```

```
m.lock()
```

```
x = 3
```

```
m.unlock()
```

```
m.lock()
```

```
x = 6
```

```
m.unlock()
```

Read/write lock allows one thread to lock for writing, and if locked for writing, no one else can lock it

OR,

one thread can lock for reading, and then lots of other threads can lock for reading